

1. Introduction

1.1. Motivation

In embedded systems development, the contribution of software engineering to the overall cost grows relative to that of hardware design [1]. As semiconductor technology advances and customer demand for broader functionality grows, an embedded application nowadays must be assembled by multiple engineering teams [2]. Each team producing independent modules that recombine into many product variants over the product family's lifetime [3]. One response to this assembly problem is to formalize it. The modules are separated from the application that consumes them, and their composition is described in a structure independent of any single module's source code. The *diff* framework, presented by Niespodziany [1], applies that idea to C++ embedded software through data-driven dependency injection: the application's composition is described in a JavaScript Object Notation (JSON) topology file consumed at startup.

While *diff* improves reusability and flexibility, composing a product variant means editing the topology file manually. The developer must ensure its structural correctness while working with limited tool support. Humble and Farley describe reliable, repeatable releases as a baseline requirement of modern software engineering [4]. Manual editing of a topology file does not meet that bar, and the gap grows with the size of the application.

Together, the industry's assembly problem and the manual authoring workflow of the *diff* framework [1] motivate ***diff-forge***: a visual tool that supports managing the dependency graph and producing a JSON file consumed by the existing C++ build pipeline without modification. This thesis extends *diff*'s data-driven composition with the missing authoring side. The next section sets out the specific gaps that *diff-forge* aims to resolve.

1.2. Problem Statement

Manual maintenance of a *diff* topology file leaves three gaps in the developer's workflow. Each grows with the size of the application and the number of developers who work with the file.

Lack of module discovery. Modules are distributed across Artifactory repositories, with many modules per repository and many versions per module. To compose a new system, the developer must learn which modules exist, what interfaces they expose, and what parameters they accept. This gap scales with the size of the catalog and the number of new systems assembled.

Lack of graph readability. The dependency graph is encoded as text. To trace which components depend on a given module, or whether one component depends on another directly or transitively, the developer reads the JSON or runs a custom query against it. Every architectural question, from the impact of a change to the presence of a cycle, requires this reconstruction.

Lack of validation. The JSON parser only checks syntax and is not capable of detecting lexical errors within each entry's configuration fields (such as identifiers, dependency references, or parameter values), or structural errors across the JSON file (cycles, unmet dependencies, interface mismatches). These errors are caught only when the binary starts, in Continuous Integration (CI), or during code review. A broken build blocks other developers, and responsibility for fixing it often falls to someone other than the original author [4].

1.3. Project Objectives

With the problem defined, the success criteria of this project follow. The tool must close each of the three gaps from the previous section, and must do so without disrupting the existing build flow.

- **Discover modules from the catalog.** At startup, the tool fetches the metadata file published with each module version from Artifactory and assembles the responses into a searchable catalog panel. Each entry displays the module's name, version, and Artifactory source.
- **Compose the graph visually.** A drag-and-drop canvas displays module instances as nodes and their dependencies as edges. The developer instantiates a module by dragging it from the catalog onto the canvas, wires dependencies by drawing edges between nodes, and edits each instance's configuration through inline fields on the node itself. The result is that the dependency graph is read and edited as a graph, not as JSON text.
- **Validate and report.** The editor validates parameter inputs, wired dependency types, and identifier references against the catalog as the developer makes changes. Per-entry errors (invalid parameter value or duplicate instance identifier) are flagged on the offending node or edge. Graph-wide errors (cycles, unmet required interfaces) surface when the user attempts to export the topology file, with feedback on every offending entry.
- **Import and export JSON topologies.** The editor imports an existing topology file so the developer can resume previous work, reports corruption or schema mismatches with the location of the offending entry, and exports the canvas state to JSON files that comply with existing build pipelines.

1.4. Thesis Structure

The remaining chapters proceed from foundations to evaluation. Chapter 2 covers the technical background: dependency injection, the *diff* framework, and related visual-composition tools. Chapter 3 analyses the problem in detail and derives the architecture drivers. Chapter 4 describes the architectural design of *diff-forge*, including technology choices and codebase layering. Chapter 5 walks through the implementation. Chapter 6 evaluates the tool against the manual JSON workflow. Chapter 7 summarises the contributions and lists directions for future work.

2. Background

This chapter establishes the technical and conceptual grounding the work rests on. Section 2.1 covers the foundations: component-based software engineering in embedded systems, the dependency injection and inversion of control patterns that the *diff* framework uses to compose systems, the *diff* framework itself, and the module distribution path from which it draws components. Section 2.2 reviews related work in visual composition for software systems and in cross-platform desktop application frameworks.

2.1. Foundations

2.1.1. Component-Based Software Engineering in Embedded Systems

Component-Based Software Engineering (CBSE) is the practice of building software from independent units called components, each with explicit interfaces that declare what it provides and what it requires from other components [5].

In an embedded system, a component is a unit like a network link driver, a timer, or a message source. Embedded products usually ship as a family of related devices that share most of their software but differ in hardware [3]. Writing a separate codebase per device does not scale. Every shared bug fix has to be applied to every codebase, and the maintenance cost grows with the number of devices in the family. CBSE supports this setting because a component, once written, can appear in any device that needs it, and each device's software is composed from a different subset of the available components [6].

CBSE on its own leaves two questions open. The first, how a component declares the other components it requires, is answered by dependency injection (§2.1.2). The second, how the assembled system resolves those declarations into concrete instances, is answered by the *diff* runtime (§2.1.3).

2.1.2. Dependency Injection and Inversion of Control

Inversion of Control (IoC) is the design principle in which a framework, rather than the component itself, decides when and how the component is instantiated and connected to its dependencies [7]. Dependency Injection (DI) implements IoC by separating the component from the construction of its dependencies: a component declares what it requires, and an external assembler supplies it, typically through constructor parameters or setter methods.

DI changes the direction of control. Without DI, a component creates its dependencies internally:

```
class Component {  
    Database db{DatabaseFactory::create()}; // self-created  
};
```

With DI, the dependencies are passed in by an external assembler:

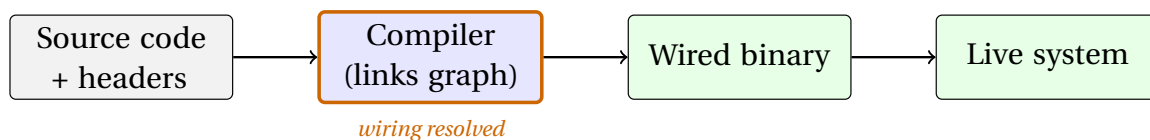
```
class Component {
```

```
public:
    Component(Database& db); // injected via constructor
private:
    Database& db_;
};
```

The same `Component` class can take a real `Database` in production, a mock one in tests, and a different implementation when wired into a separate product variant.

Static vs. Runtime DI DI has two main forms, distinguished by when the wiring is resolved (Figure 2.1).

Static DI



Runtime DI

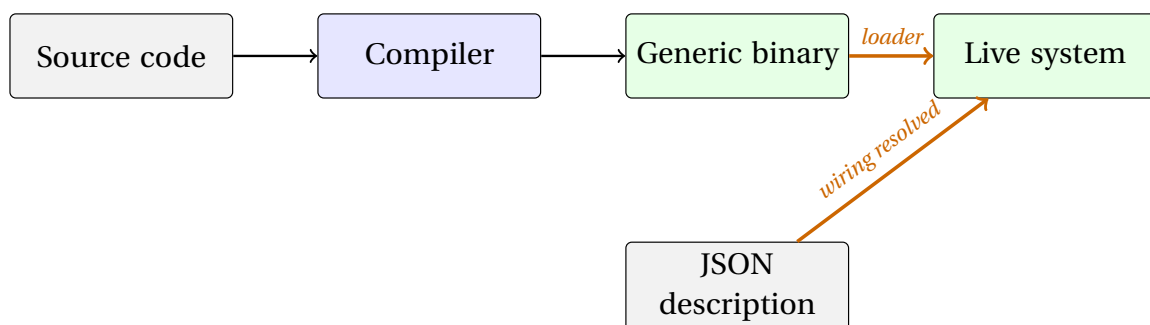


Figure 2.1. Static DI vs. runtime DI.

Static DI resolves wiring at compile time: a header or template specialisation names the concrete type for each interface, the compiler links the graph, and the resulting binary has no wiring step at startup. The compiler rejects unfilled dependencies, so the system is verifiable. The binary contains exactly one topology, so every variant requires a rebuild.

Runtime DI resolves wiring at process startup from a data description (XML, JSON, YAML). The same binary takes different descriptions and produces different systems, so one build serves multiple variants. The compiler no longer checks the wiring, so a malformed description fails only when the runtime loader executes it.

DI in C++14 relies on template metaprogramming and static initialisation. C++ has no reflection-based DI container like Java or C#, so frameworks build their own using the Curiously Recurring Template Pattern [8]. C++14 [9] adds variadic templates and stronger `constexpr` support that simplify this construction. Niespodziany [1] details how the *diff* framework (§2.1.3) applies these techniques.

2.1.3. The *diff* Framework

The *diff* framework [1] is a runtime DI implementation for C++14 embedded software. Figure 2.2 sketches its end-to-end workflow: component classes are compiled into the binary and register themselves before `main()` runs, then a JSON topology file drives the construction of a live system. The examples below come from *diff_breaker*, the reference application that ships with the framework.

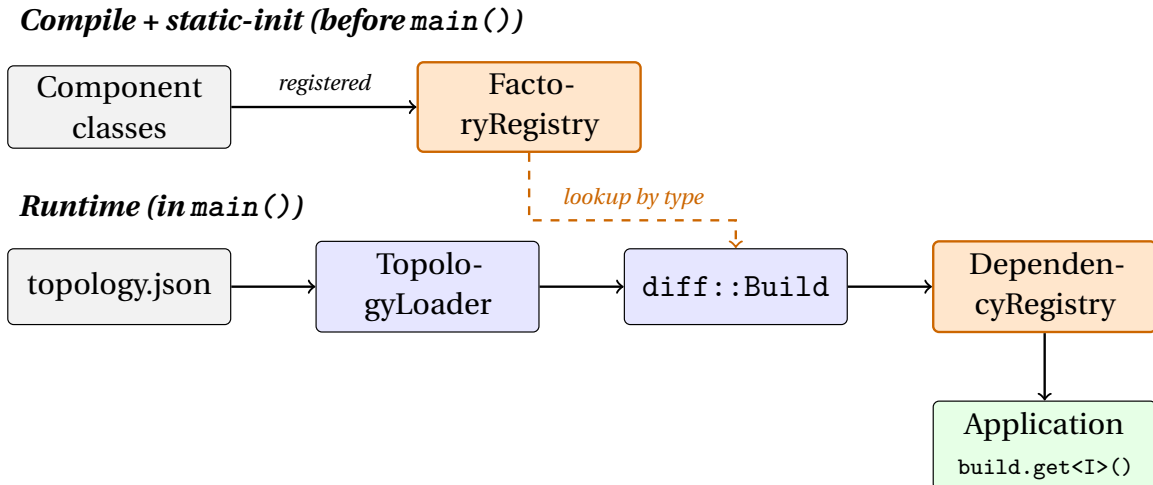


Figure 2.2. The *diff* workflow. Component classes register their factories at static-initialisation time. At runtime, the framework reads the topology JSON, looks up the factories by type name, and instantiates the components into a live application accessible by interface.

Components and interface components Niespodziany [1] distinguishes two kinds of components. A regular *component* implements algorithms and data structures. An *interface component* is header-only: a pure-virtual C++ class that declares an interface and lives in its own translation unit so no implementation can leak detail across the contract. A component class inherits from `diff::Component<T, ...>`, where `T` is the class itself and the remaining template parameters declare which interfaces the component provides. Listing 1 shows `LinkGsm`: it implements `ILink` and exposes itself under that interface, so any later component that needs an `ILink` can be wired to it.

Listing 1. A *diff* component that implements the `ILink` interface.

```
// ILink.h -- interface component (header-only)
class ILink {
public:
    virtual ~ILink() = default;
    virtual bool send(const std::string& message) = 0;
};

// LinkGsm.h
class LinkGsm : public diff::Component<LinkGsm, diff::as<ILink>> {
```

```
public:
    LinkGsm();
    bool send(const std::string& message) override;
};
```

The topology file The system's wiring is described by a JSON *topology* file. Each entry carries the component's type name, a unique instance id, the ordered ids of the dependencies that should be injected into the constructor, and a typed configuration map (Listing 2). The framework instantiates the entries in order, so the topology must be in dependency order: every component appears after all the components it depends on.

Listing 2. Topology entry consumed by the *diff* loader at startup.

```
{
  "type": "MessageSource",
  "id": "msgSrc0",
  "version": "1.0.0",
  "source": "https://artifactory.example.com/artifactory/diff",
  "dependencies": ["linkEth0", "linkGsm0"],
  "config": {
    "count": 3,
    "content": "Diff says hello."
  }
}
```

Topology as a data product The same compiled binary, given two different topology files, produces two different running systems with different components and different wiring. This is what Niespodziany [1] calls *data-driven* composition. A new variant is a JSON edit, not a code change. The trade-off is that the JSON file becomes the primary artifact of the system, and any error in it shows up only when the loader runs. That trade-off is what motivates the visual editor presented in this thesis.

2.1.4. Module Distribution: Conan and Artifactory

C++ has no standard package manager, so teams pick one of several third-party tools to manage binary dependencies. **Conan** [10] is the one used by the *diff* ecosystem: a package manager that builds and consumes C++ libraries through *recipes*, build scripts that describe how to compile, package, and install a library. A Conan package is identified by name/version@user/channel and may carry pre-built binaries for multiple compilers, architectures, and build types.

Artifactory [11] is a binary repository manager from JFrog that, among other formats, supports the Conan protocol. Teams publish their internal C++ libraries as Conan packages to an Artifactory repository, and consumer projects fetch them through standard Conan tooling. In a *diff*-based environment, each component is a Conan package, and each

package carries a catalog entry that describes the component to potential consumers: the interfaces it implements, the dependencies it requires, and the configuration parameters it accepts. Listing 3 shows the catalog entry for a `MessageSource` component.

Listing 3. Catalog entry for the `MessageSource` component, published alongside its Conan package.

```
{
  "type": "MessageSource",
  "version": "1.0.0",
  "implements": ["IProcessable"],
  "requires": [
    { "slot": "link",    "type": "ILink" },
    { "slot": "backup",  "type": "ILink", "isArray": true }
  ],
  "config": {
    "count": { "type": "uint32", "min": 1, "max": 1000 },
    "content": { "type": "string", "default": "msg" }
  }
}
```

The catalog entry is a *schema* rather than an instance: `implements` lists the interfaces the component provides, `requires` describes the slots that must be filled by other components (each with a name, an interface type, and an `isArray` flag for multi-binding slots), and `config` declares each parameter's type, allowed range, and default value. A topology entry (§2.1.3) is a concrete *instance* of a catalog entry: it picks an `id`, fills each `requires` slot with the `id` of another instance, and assigns concrete values inside `config`.

For *diff-forge*, the catalog of available modules is the set of catalog entries hosted on Artifactory. At startup, the tool queries the repository's Conan API, downloads each component's catalog entry, and presents the entries in the side panel. The ingestion pipeline is described in §4.3.

2.2. Related Work

2.2.1. Visual Composition for Software Systems

Visual composition tools let developers build software by arranging and connecting nodes on a canvas, replacing or supplementing textual code. The approach has a long history in domain-specific environments. LabVIEW [12] uses a dataflow paradigm for instrumentation and measurement software. Simulink [13] applies it to control-system modelling. Node-RED [14] offers a flow-based editor for IoT devices and web services. Unreal Engine's Blueprint system [15] exposes game-engine APIs as a visual scripting language.

In all of these tools, the canvas is a directed graph. Nodes represent units of computation or component instances. Edges represent dataflow or dependency. The two graph-theory

concepts relevant to *diff-forge* are the directed acyclic graph and the algorithm for ordering its nodes.

Directed Acyclic Graph (DAG) is a finite directed graph that contains no directed cycles [16]. Each edge has a direction, and no sequence of edges returns to its starting node. DAGs are the natural model for dependency relationships in software. Each component depends on zero or more others, and a cycle would mean a component transitively depends on itself. A topology with a cycle cannot be instantiated, because no component in the cycle has all of its dependencies available.

The *diff* framework requires the topology to be a DAG. The loader refuses to instantiate a topology that contains a cycle. *diff-forge* enforces the DAG property at export time. When the user attempts to export a topology that contains a cycle, the validation engine rejects it and points at the cycle's edges.

Topological Sorting produces a linear ordering of a DAG's nodes such that for every directed edge (u, v) , u comes before v [16]. For a dependency graph, a topological order is a valid construction sequence. Every component's dependencies appear before the component itself.

Kahn's algorithm [17] computes a topological order in linear time (Figure 2.3). It begins with the set of nodes that have no incoming edges. At each step, it removes one such node from the set, appends it to the output, and reduces the in-degree of each neighbour. Any neighbour that drops to zero in-degree joins the set. The algorithm terminates when the set is empty. If all nodes have been emitted, the output is a valid topological order. If some nodes remain unemitted, the graph contains a cycle.

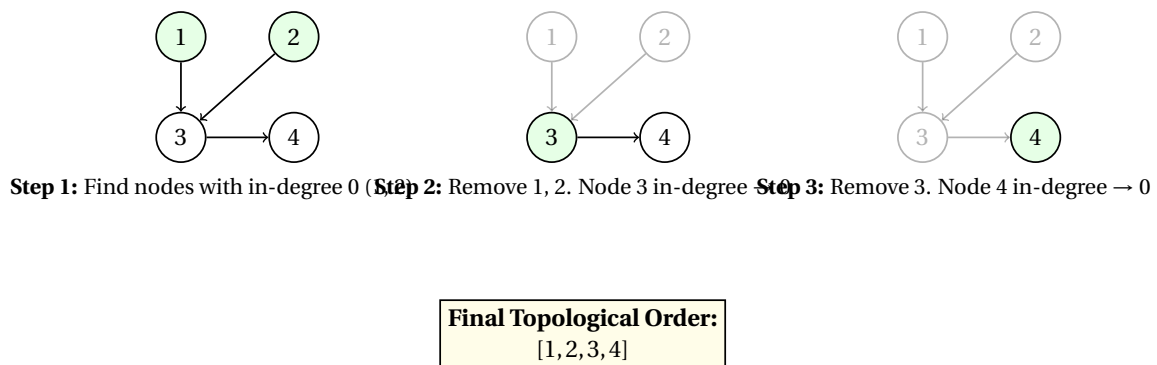


Figure 2.3. Step-by-step topological sorting using Kahn's algorithm.

diff-forge uses Kahn's algorithm during export. The topology JSON file requires entries to appear in load-compatible order (leaves before roots). Kahn's algorithm produces this order from the canvas graph (§4.5).

2.2.2. Cross-Platform Desktop Application Frameworks

A cross-platform desktop application framework lets a single codebase target Windows, macOS, and Linux without per-platform rewrites. The candidates differ along three dimensions. The first is the rendering technology, either a web stack or native widgets. The

second is the language used for application logic. The third is the deployment profile, such as binary size and memory footprint. Table 2.1 summarises the most active frameworks at the time of writing.

Table 2.1. Cross-platform desktop application frameworks compared. Figures are typical for a minimal application bundled for production distribution.

Framework	Renderer	App Language	Binary Size	Memory Footprint
Electron	Chromium + Node.js	TypeScript / JavaScript	100–200 MB	200–500 MB
Tauri	Native WebView	Rust + TypeScript	10–30 MB	50–150 MB
Qt (Widgets)	Native widgets	C++	40–100 MB	50–200 MB
.NET MAUI	Native + WebView	C# / XAML	50–100 MB	100–300 MB
Flutter Desktop	Skia	Dart	30–60 MB	80–200 MB

Electron [18] bundles a Chromium browser and a Node.js runtime. The application UI is written as a standard web page, and the Node.js backend provides access to OS-level APIs. The two-process architecture (main process plus renderer process) and the IPC layer between them are described in §5.2. The trade-off is binary size and memory footprint, both larger than native alternatives.

Tauri [19] uses the OS-provided WebView (WebKit on macOS, WebView2 on Windows, WebKitGTK on Linux) for the UI and Rust for the backend. This achieves an order-of-magnitude reduction in binary size and memory at the cost of a less uniform rendering surface across platforms.

Qt [20] is the long-standing C++ framework for native desktop applications. The Widgets module produces compact, native-looking applications. The QML/Quick module offers a declarative UI with hardware acceleration. The trade-off is that the application logic is written in C++, which limits the available ecosystem of UI libraries compared to the web stack.

.NET MAUI [21] is Microsoft’s cross-platform framework. It combines native widgets and embedded WebViews under a C#/XAML programming model. Its primary target is mobile (Android and iOS) with desktop support as a secondary use case.

Flutter Desktop [22] uses Google’s Skia renderer to draw all UI elements directly. The result is a consistent look across platforms at the cost of not matching native conventions. Its primary language is Dart.

For *diff-forge*, Electron with React and TypeScript was selected (§4.1). Familiarity with the web stack and the maturity of the React Flow canvas library [23] outweighed the binary-size cost.

References

- [1] S. Niespodziany, “Data-driven dependency injection for embedded software, enhancing reusability with c++”, *International Journal of Electronics and Telecommunications*, vol. 71, no. 4, pp. 1–7, 2025. DOI: 10.24425/ijet.2025.155469.
- [2] P. C. Pendharkar and J. A. Rodger, “The relationship between software development team size and software development cost”, *Communications of the ACM*, vol. 52, no. 1, pp. 141–144, 2009. DOI: 10.1145/1435417.1435449.
- [3] P. Clements and L. Northrop, *Software Product Lines: Practices and Patterns*. Addison-Wesley Professional, 2001, ISBN: 0-201-70332-7.
- [4] J. Humble and D. Farley, *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley Professional, 2010.
- [5] C. Szyperski, D. Gruntz, and S. Murer, *Component Software: Beyond Object-Oriented Programming*, 2nd ed. Addison-Wesley Professional, 2002, ISBN: 0-201-74572-0.
- [6] I. Crnkovic and M. Larsson, *Building Reliable Component-Based Software Systems*. Artech House, 2002, ISBN: 1-58053-327-2.
- [7] M. Fowler. “Inversion of control containers and the dependency injection pattern”. (2004), [Online]. Available: <https://martinfowler.com/articles/injection.html> (visited on 05/11/2026).
- [8] J. O. Coplien, “Curiously recurring template patterns”, *C++ Report*, vol. 7, no. 2, pp. 24–27, 1995.
- [9] *Isol/iec 14882:2014 programming languages – c++*, International Organization for Standardization, Geneva, Switzerland, 2014.
- [10] Conan Authors. “Conan, the c and c++ package manager: Documentation”. (2025), [Online]. Available: <https://docs.conan.io/> (visited on 05/15/2026).
- [11] JFrog. “Jfrog artifactory documentation”. (2025), [Online]. Available: <https://jfrog.com/help/r/jfrog-artifactory-documentation> (visited on 05/15/2026).
- [12] National Instruments. “Labview documentation”. (2025), [Online]. Available: <https://www.ni.com/en/support/documentation/supplemental/08/labview.html>.
- [13] MathWorks. “Simulink documentation”. (2025), [Online]. Available: <https://www.mathworks.com/help/simulink/>.
- [14] OpenJS Foundation. “Node-red documentation”. (2025), [Online]. Available: <https://nodered.org/docs/>.
- [15] Epic Games. “Unreal engine blueprints visual scripting”. (2025), [Online]. Available: <https://dev.epicgames.com/documentation/en-us/unreal-engine/blueprints-visual-scripting-in-unreal-engine>.
- [16] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. MIT Press, 2009.
- [17] A. B. Kahn, “Topological sorting of large networks”, *Communications of the ACM*, vol. 5, no. 11, pp. 558–562, 1962. DOI: 10.1145/368996.369025.
- [18] O. Foundation. “Electron documentation”. (2024), [Online]. Available: <https://www.electronjs.org/docs/latest>.

2. References

- [19] Tauri Programme within The Commons Conservancy. "Tauri documentation". (2025), [Online]. Available: <https://tauri.app/start/>.
- [20] The Qt Company. "Qt documentation". (2025), [Online]. Available: <https://doc.qt.io/>.
- [21] Microsoft. ".NET multi-platform app ui (.NET MAUI) documentation". (2025), [Online]. Available: <https://learn.microsoft.com/en-us/dotnet/maui/>.
- [22] Google. "Flutter desktop documentation". (2025), [Online]. Available: <https://docs.flutter.dev/platform-integration/desktop>.
- [23] xyflow. "React flow: A library for building node-based editors and diagrams". (2025), [Online]. Available: <https://reactflow.dev/>.

List of Symbols and Abbreviations

- WUT** – Warsaw University of Technology
- EiTI** – Faculty of Electronics and Information Technology
- IAiIS** – Institute of Control and Computation Engineering
- JSON** – JavaScript Object Notation
- CI** – Continuous Integration
- CBSE** – Component-Based Software Engineering
- IoC** – Inversion of Control
- DI** – Dependency Injection
- CRTP** – Curiously Recurring Template Pattern
- DAG** – Directed Acyclic Graph
- IPC** – Inter-Process Communication

List of Figures

- 2.1 Static DI vs. runtime DI. 4
- 2.2 The *diff* workflow. Component classes register their factories at static-initialisation time. At runtime, the framework reads the topology JSON, looks up the factories by type name, and instantiates the components into a live application accessible by interface. 5
- 2.3 Step-by-step topological sorting using Kahn’s algorithm. 8

List of Tables

- 2.1 Cross-platform desktop application frameworks compared. Figures are typical for a minimal application bundled for production distribution. 9

List of Appendices